

Introducción

Salma Fonseca Curbelo

El tiempo y los recursos son elementos finitos en cualquier escenario organizacional. Esto hace que la planificación y la coordinación de eventos dejen de ser una formalidad para convertirse en una necesidad. Es por esto que los horarios cobran importancia: son las estructuras que sostienen el funcionamiento diario de las instituciones y evitan el colapso en sus operaciones.

La asignación de horarios constituye un problema de planificación de la actualidad [1]. Este proceso consiste en organizar y distribuir un conjunto de actividades en determinados intervalos de tiempo, teniendo en cuenta múltiples restricciones como la disponibilidad de recursos, espacios y personas, con el fin de evitar conflictos y solapamientos [2]. Sin una correcta gestión de estos elementos, la operatividad y logística de cualquier institución o medio de comunicación colapsaría [3].

Dentro del dominio de la confección de horarios, el aspecto más crítico es tener algoritmos capaces de encontrar soluciones factibles en tiempos computacionales razonables y minimizar las violaciones de restricciones blandas. Los problemas de asignación de horarios académicos y televisivos se caracterizan por una elevada complejidad combinatoria: al aumentar las actividades, los recursos y las restricciones, el espacio de soluciones crece de forma exponencial [4]. Por esta razón, la mayoría de las variantes del problema de asignación de horarios (Timetabling) se clasifican como problemas NP-duros, lo que impide que los métodos exactos encuentren soluciones óptimas en tiempos razonables y hace necesario el uso de metaheurísticas para explorar el espacio de soluciones [3].

En esta investigación, el estudio se centra específicamente en dos tipos de escenarios: la planificación de horarios académicos, como los que se confeccionan en una facultad de una universidad, y la programación de parrillas televisivas, debido a sus diferencias en restricciones, objetivos y criterios de evaluación [2, 5]. En la literatura científica, se han propuesto algoritmos y heurísticas para abordar este problema. En el ámbito de los horarios de facultad, investigaciones

han demostrado la utilidad de las reglas de manejo de restricciones (CHR) para procesar requerimientos [6], aunque en la actualidad las soluciones se inclinan hacia esquemas híbridos y plataformas interactivas como UniTime, que asisten al planificador humano [7]. Paralelamente, en el contexto de la facultad de Matemática y Computación de la Universidad de La Habana (MATCOM), se ha transitado desde la programación entera hacia el modelado orientado a la evaluación (un enfoque donde las restricciones no prohíben estrictamente una solución, sino que penalizan su función objetivo), maximizando la “felicidad” (una métrica que cuantifica el grado de satisfacción de las preferencias de profesores y estudiantes) de los horarios mediante métodos metaheurísticos [8]. Por otro lado, en la programación de televisión se ha evolucionado desde enfoques clásicos de Programación Lineal Entera Mixta (MILP) [5] hasta la aplicación de Inteligencia Artificial predictiva y Aprendizaje por Refuerzo Profundo [9] para decidir la posición de los programas en la parrilla de forma autónoma.

A pesar de estos avances, gran parte de las herramientas y plataformas especializadas presentan arquitecturas cerradas, lo que dificulta extraer su motor de evaluación para integrarlo en entornos tecnológicos flexibles o adaptables [10]. El objeto de estudio de esta investigación se enmarca en este contexto. La investigación transita desde el área general de la planificación combinatoria y la optimización hacia el modelado de restricciones orientado a la evaluación. Finalmente, se enfoca en el uso de Lenguajes Específicos de Dominio (DSLs), Árboles de Sintaxis Abstracta (AST) y la generación de código para asistir la creación de horarios complejos. Teniendo en cuenta todo lo anterior, surge la siguiente pregunta científica: ¿Es posible, mediante el diseño de un Lenguaje de Dominio Específico (DSL) declarativo, abstraer la formalización de las reglas y restricciones en los problemas de gestión de horarios para generar automáticamente algoritmos de evaluación de soluciones polimórficos, es decir, capaces de ser traducidos y ejecutados en distintos lenguajes de programación o entornos tecnológicos?

Para sustentar esta aproximación tecnológica, el trabajo se apoyará en el *Common Lisp Object System* (CLOS). Este es un sistema orientado a objetos integrado en Lisp que proporciona mecanismos avanzados como la herencia múltiple, el despacho dinámico y las funciones genéricas, siendo la herramienta estructural clave que permitirá manipular las entidades del problema y facilitar la generación de código.

El objetivo general de este trabajo es diseñar e implementar un Lenguaje

de Dominio Específico (DSL) en Common Lisp que permita describir las reglas y restricciones de problemas de gestión de horarios, con el fin de generar automáticamente algoritmos de evaluación de soluciones en múltiples lenguajes de programación.

Para el cumplimiento del objetivo general, se han trazado los siguientes objetivos específicos:

- Analizar el dominio de los problemas de gestión de horarios para identificar y formalizar las restricciones y criterios de evaluación comunes.
- Diseñar la sintaxis y la semántica de un DSL declarativo que permita especificar no solo las entidades del problema sino también las reglas para evaluar la calidad de una solución.
- Implementar el núcleo del sistema en Common Lisp, procesando las descripciones del DSL para construir una representación intermedia y abstracta de la función de evaluación y del conjunto de restricciones aplicables.
- Desarrollar un sistema de generación de código polimórfico basado en CLOS, que traduzca la representación intermedia de la función de evaluación a código fuente ejecutable en distintos lenguajes de programación.

Para dar solución a esta problemática, se propone construir un DSL interno aprovechando la flexibilidad y la propiedad homoicónica de Common Lisp [11], lo cual permite manipular su código fuente como datos. Mediante el uso de macros [12], se definen nuevas abstracciones sintácticas (elementos del lenguaje que ocultan la complejidad matemática subyacente y permiten escribir reglas mediante palabras clave propias del dominio) que el usuario emplea para declarar las reglas y restricciones. Al evaluar directamente los constructores de estas clases durante el procesamiento del DSL, se instancia un Árbol de Sintaxis Abstracta (AST), sin necesidad de un paso de transformación intermedio. Posteriormente, gracias al sistema de funciones genéricas y despacho dinámico de CLOS (un mecanismo donde el método específico a invocar se determina en tiempo de ejecución según el tipo de los argumentos), el sistema traduce este AST a código ejecutable, permitiendo así generar el evaluador de horarios en cualquier lenguaje deseado.

Como resultado fundamental de esta investigación, se obtiene un sistema que permite obtener, a partir de una descripción de un horario en el DSL creado, funciones para determinar cuán conveniente es un horario dado.

La estructura de este documento es la siguiente: En el Capítulo 1, “Fundamentos y estado del arte de la confección de horarios”, se abordan los fundamentos y el estado del arte de la asignación de horarios, analizando las particularidades del entorno universitario y televisivo, así como el uso de metaheurísticas para su resolución. El Capítulo 2, “Fundamentos de Lenguajes y Representación”, presenta los fundamentos teóricos sobre los Lenguajes Específicos de Dominio (DSLs), la generación de código mediante Árboles de Sintaxis Abstracta (AST) y las capacidades de metaprogramación en Common Lisp. El Capítulo 3, “Diseño e Implementación del DSL”, detalla la sintaxis declarativa propuesta y la arquitectura subyacente en Lisp para la instanciación de las entidades y restricciones. El Capítulo 4, “Traducción Polimórfica y Validación”, expone el mecanismo de generación de código fuente mediante CLOS y muestra los resultados de la validación del sistema. Finalmente, el Capítulo 5 presenta las “Conclusiones” y recomendaciones derivadas de la investigación.

References

- [1] Wren, A. (1996). *Scheduling, Timetabling and Rostering—A Special Relationship?* In Practice and Theory of Automated Timetabling (PATAT 1995), Springer, pp. 46-75.
- [2] Schaerf, A. (1999). *A survey of automated timetabling*. Artificial Intelligence Review, 13(2), pp. 87-127.
- [3] Burke, E. K., & Petrovic, S. (2002). *Recent research directions in automated timetabling*. European Journal of Operational Research, 140(2), pp. 266-280.
- [4] Karp, R. M. (1972). *Reducibility among combinatorial problems*. In Complexity of Computer Computations, Springer, pp. 85-103.
- [5] García-Villoria, A., & Pastor, R. (2011). *A MILP model for the television break scheduling problem*. European Journal of Operational Research, 212(1), pp. 165-174.
- [6] Abdennadher, S., & Marte, M. (2000). *University timetabling using constraint handling rules*. Applied Artificial Intelligence, 14(4), pp. 311-325.
- [7] Müller, T. (2010). *UniTime: Distributed architecture for university timetabling*. In Practice and Theory of Automated Timetabling (PATAT 2010).
- [8] Travieso Sosa, E. (2012). *Modelo de restricciones para un sistema de generación automatizada de horarios*. Facultad de Matemática y Computación, Universidad de La Habana.
- [9] Zhou, Z., et al. (2020). *Deep Reinforcement Learning for Autonomous Content Scheduling*. IEEE Transactions on Neural Networks and Learning Systems.
- [10] McCollum, B., et al. (2012). *A framework for modelling and evaluating academic timetabling problems*. Annals of Operations Research, 194(1), pp. 305-322.
- [11] Steele, G. L. (1990). *Common Lisp the Language* (2nd ed.). Digital Press.
- [12] Norvig, P. (1992). *Paradigms of Artificial Intelligence Programming: Case Studies in Common Lisp*. Morgan Kaufmann.